

密码分析学实验报告

姓名：邱泽辉、江李阳、王健一

学号：202400460046、202400460042、202400460037

专业：密码科学与技术

日期：October 29, 2026

Contents

1 实验环境 **2**

2 实验原理 **2**
2.1 线性近似表达式 2
2.1.1 线性近似表达式的概率 2
2.2 线性路线 2
2.3 线性近似表 2

3 实验内容 **3**
3.1 任务一 3
3.2 任务二 4
3.2.1 考量: 活跃 S 盒的个数 4
3.2.2 贪心算法 6
3.2.3 意外之喜 8
3.3 任务三 11
3.3.1 $\alpha_L \neq \beta_R$ 12
3.3.2 $\alpha_L = \beta_R$ 12

4 实验报告完整地址 **15**

5 AI 使用情况 **15**

1 组员分工

- 邱泽辉：完成任务二、三的代码和实验报告对应章节内容的撰写。
- 江李阳：完成任务一代码和实验报告对应章节内容的撰写，修改润色实验报告。
- 王健一：完成任务二、三的代码和实验报告对应章节内容的撰写，修改润色实验报告。

2 实验环境

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约为 2.1 GHz；内存 16 GB。
- 软件环境：Windows、MacOS,Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。

3 实验原理

3.1 线性近似表达式

针对密码算法 $E_K(\cdot)$，对于一对给定的线性掩码 (α, β) 可视为 $E_K(\cdot)$ 的一个线性近似，并称

$$\alpha \cdot x \oplus \beta \cdot E_K(x)$$

为 $E_K(\cdot)$ 的一个线性近似表达式，其中 α 和 β 分别被称为线性近似的输入掩码和输出掩码。

3.1.1 线性近似表达式的概率

依据上述对线性近似表达式的定义，给定一个线性近似 (α, β)，相应的线性近似表达式的概率 $Pr(\alpha, \beta)$ 为使得线性近似表达式取 0 的输入在整个输入空间中的占比，对应如下计算式子：

$$Pr(\alpha, \beta) = \frac{\#\{X \in \mathbb{F}_2^n | \alpha \cdot X \oplus \beta \cdot E_K(X) = 0\}}{2^n}$$

其中定义线性近似的偏差为线性近似表达式的概率与 $1/2$ 的差值，即

$$\varepsilon(\alpha, \beta) = Pr(\alpha, \beta) - \frac{1}{2}$$

3.2 线性路线

对于迭代型分组密码算法 $E_K(\cdot)$，设输入和输出掩码分别为 α、β，中间状态为 $Y_1, Y_2, \cdots, Y_{r-1}$ 处掩码分别为 $\alpha_1, \alpha_2, \cdots, \alpha_{r-1}$，则称 $(\alpha_1, \alpha_2, \cdots, \alpha_{r-1}, \beta)$ 为 $E_K(\cdot)$ 的一条线性路线。注意到这里线性近似与线性路线是不同的概念，其中线性近似中包含有多条线性路线（对应中间状态的不同掩码）。

3.3 线性近似表

线性近似表是记录对于给定 S 盒的所有线性近似的偏差信息，其第 α 行第 β 列为线性近似掩码为 (α, β) 的线性近似值为 0 的输出个数与 2^{m-1} 的差，其中 m 为 S 盒的输入位数。具体来说，对于 $m \times n$ 的 S 盒，线性近似表为一张包含 2^m 行、2^n 列的表，表格中第 α 行，第 β 列的元素为

$$\#\{(x, S(x)) \in \mathbb{F}_2^m \times \mathbb{F}_2^n | \alpha \cdot x \oplus \beta \cdot S(x) = 0\} - 2^{m-1}$$

其中 $\alpha \in \{0, 1, ..., 2^m - 1\}, \beta \in \{0, 1, ..., 2^n - 1\}$, 将表中元素除以 2^m 即为相应的线性近似的偏差。

4 实验内容

4.1 任务一

题目给出的 S 盒输入大小 $m = 4$bit，输出大小 $n = m = 4$bit, 所以 S 盒的线性近似表大小就是 $2^4 \times 2^4$, 里面每一项记作 $N(\alpha, \beta)$ 先遍历输入掩码 $\alpha \in \mathbf{F}_2^4$，再遍历输出掩码 $\beta \in \mathbf{F}_2^4$，再遍历所有 2^4 种输入 x，计算对应线性近似式

$$\alpha \cdot x \oplus \beta \cdot S(x)$$

统计其中为 0 的个数 $N(\alpha, \beta)$，然后在 S 盒的 LAT 中赋值 $N(\alpha, \beta) - 2^3$。

记 parity(v) 为 v 中 1 的个数的奇偶性，& 表示按位与运算, 这里之所以要用 parity 运算，是因为实现内积是计算较为复杂的，在我们按位与之后需要把每一位异或起来。这里注意到，内积的结果不是 0 就是 1，因此只要统计有多少个 1 就行了，即有偶数个 1 最终异或结果就是 0，有奇数个 1 最终异或结果就是 1。对应伪代码如算法 1所示。

算法 1 4 比特 S 盒线性近似表的计算

输入： 4 比特 S 盒 $S[0..15]$
输出： 线性近似表 LAT$[0..15, 0..15]$

```
for α = 0, ..., 15 do
    for β = 0, ..., 15 do
        N ← 0
        for x = 0, ..., 15 do
            p ← parity(α&x)
            q ← parity(β&S[x])
            if p = q then
                N ← N + 1
            end if
        end for
        LAT[α, β] ← N − 8
    end for
end for
return LAT
```

代码运行结果如图 1所示。

```
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab3\code> uv run python task1.py
        0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
 0 :    8  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
 1 :    0  0  0  0 -2 -2  2  2  0  0  0  0  2  2 -2  6
 2 :    0  0 -4  0  4  0  0  0 -4  0  0  0  0  0 -4  0
 3 :    0 -4  0  0 -2 -2  2 -2  0  0  0  4  2 -2 -2 -2
 4 :    0 -2  2  0  4  2  2  0  4 -2 -2  0  0  2 -2  0
 5 :    0 -2  2  0  2  0 -4  2  0  2  2  4 -2  0  0  2
 6 :    0  2  2  0  0 -2 -2  0  0  2 -6  0  0 -2 -2  0
 7 :    0 -2 -2  0  2 -4  0 -2  0 -2 -2  0 -2  0  4  2
 8 :    0  0  4  0  2 -2  2  2 -4  0  0  0  2  2  2 -2
 9 :    0  4  0  0  0  0  0 -4  0  0  0  4  0  4  0  0
 A :    0  0  0  0 -2 -2  2  2  0  0  0  0 -6  2 -2 -2
 B :    0  0  0 -8  0  0  0  0  0  0  0  0  0  0  0  0
 C :    0  2  2  0  2  0  4 -2  0  2  2  0 -2 -4  0  2
 D :    0 -2 -2  0  0  2  2  0  0  6 -2  0  0  2  2  0
 E :    0 -2  2  0 -2  4  0 -2 -4 -2 -2  0 -2  0  0  2
 F :    0 -2  2  0  0 -2 -2 -4  0  2  2 -4  0  2 -2  0
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab3\code>
```

Figure 1: 任务一计算得到的 S 盒线性近似表

其中偏差最大的就是 $\texttt{0x0}B \to \texttt{0x03}$，对应偏差为：

$$\varepsilon(\texttt{0x0}B, \texttt{0x03}) = \frac{-8}{16} = -\frac{1}{2}$$

4.2 任务二

本任务基于任务一计算得到的 S 盒线性近似表（LAT），搜索一条 4 轮迭代线性路线并计算其偏差。

4.2.1 活跃 S 盒个数的考量

CipherFour 算法包含四个 S 盒，P 置换使不同 S 盒的比特相互影响，增加了路线分析的复杂度。堆积引理和偏差分析建立在 Markov 假设之上，需要关注相关变量的独立性。因此，先考虑限制活跃 S 盒的数量，以简化分析，具体要求如图 2所示。

- 堆积引理的独立性要求
  - 堆积引理成立的关键在于诸个X_i的独立性
  - Matsui对DES的分析中，每一轮的线性近似式只含有1个活跃S盒（输出掩码非零）。若包含相邻的2个以上活跃S盒，利用堆积引理获得最终的线性近似式其偏差值将明显偏离真实值，则密钥信息不再准确

Figure 2: 堆积引理要求

采用相关度表示时，堆积引理的形式更为简洁。考虑输入掩码 $\alpha = (\alpha_0, \alpha_1, \alpha_2, \alpha_3)$ 和输出掩码 $\beta = (\beta_0, \beta_1, \beta_2, \beta_3)$，4 个 S 盒对应的一轮 S 层相关度为

$$c(\alpha, \beta) = \prod_{j=0}^{3} c(\alpha_j, \beta_j)$$

相关度为偏差的 2 倍，相关度较大的路线更有利于线性分析。由于 $c(0,0) = 1$，可以优先考虑活跃 S 盒较少的路线。不过，为避免全零掩码对应的平凡情况，路线中至少需要一个活跃 S 盒。

不过，除 $c(0,0) = 1$ 外，还需要考虑 LAT 中的其他掩码组合。前面的 LAT 中，除了 (0,0) 对应的表项为 8 外，任务一还找到了 $B \to 3$ 这一组合，如下图所示。

```
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab3\code> uv run python task1.py
      0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
 0 :  8  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
 1 :  0  0  0  0 -2 -2  2  2  0  0  0  0  2  2 -2  6
 2 :  0  0 -4  0  4  0  0  0 -4  0  0  0  0  0 -4  0
 3 :  0 -4  0  0 -2 -2  2 -2  0  0  0  4  2 -2 -2 -2
 4 :  0 -2  2  0  4  2  2  0  4 -2 -2  0  0  2 -2  0
 5 :  0 -2  2  0  2  0 -4  2  0  2  2  4 -2  0  0  2
 6 :  0  2  2  0  0 -2 -2  0  0  2 -6  0  0 -2 -2  0
 7 :  0 -2 -2  0  2 -4  0 -2  0 -2 -2  0 -2  0  4  2
 8 :  0  0  4  0  2 -2  2  2 -4  0  0  0  2  2  2 -2
 9 :  0  4  0  0  0  0  0 -4  0  0  0  4  0  4  0  0
 A :  0  0  0  0 -2 -2  2  2  0  0  0  0 -6  2 -2 -2
 B :  0  0  0 -8  0  0  0  0  0  0  0  0  0  0  0  0
 C :  0  2  2  0  2  0  4 -2  0  2  2  0 -2 -4  0  2
 D :  0 -2 -2  0  0  2  2  0  0  6 -2  0  0  2  2  0
 E :  0 -2  2  0 -2  4  0 -2 -4 -2 -2  0 -2  0  0  2
 F :  0 -2  2  0  0 -2 -2 -4  0  2  2 -4  0  2 -2  0
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab3\code>
```

Figure 3: LAT

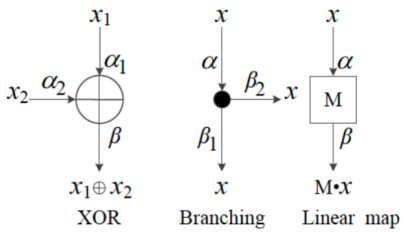
也就是说 $c(B, 3) = 1$，从相关度看，该组合与 (0,0) 具有相同的效果，理论上也可以选用。但本实验还包含 P 层置换，引入该组合会增加后续分析的复杂度。为便于人工分析，先采用较简单的路线，选择 (0,0) 作为非活跃 S 盒的掩码，从仅有一个活跃 S 盒的情形开始讨论。

4.2.2 贪心搜索思路

与差分路线分析相比，线性路线分析还需要考虑线性变换对掩码的影响，如下图所示。

线性变换的掩码传播情况——以概率1过线性变换

- 异或运算（XOR）
  - 输入掩码分别为a_1、a_2，输出掩码为β，则$a_1 = a_2 = \beta$
- 分支运算（Branching）
  - 输入掩码为a，输出掩码分别为β_1、β_2，则$\beta_1 \oplus \beta_2 = a$
- 线性映射（Linear Map）
  - 输入掩码为a，输出掩码为β，则$a = M^T\beta$（以AES的矩阵乘为例证明，留作练习）



线性掩码需要经过 P 置换。将该置换表示为矩阵，并仍记作 P，可按下图的方法构造该矩阵。

```
sage: P = [0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15]
....:
....: M = [[0] * 16 for _ in range(16)]
....:
....: for i, j in enumerate(P):
....:     M[j][i] = 1
....:
....: for row in M:
....:     print(row)
....:
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
[sage:
sage:
```

设第一轮的输入掩码为 α_1，输出掩码为 β_1。若不考虑 P 置换，应满足 $\beta_1 = \alpha_2$；引入 P 置换后，则应满足 $\alpha_2 = P\beta_1$。为明确掩码的传播过程，先考虑如下初始状态：

$$(\alpha_{11}, 0, 0, 0) \xrightarrow{S} (\beta_{11}, 0, 0, 0) \xrightarrow{P} (\alpha_{21}, \alpha_{22}, \alpha_{23}, \alpha_{24})$$

进一步得到如下四轮传播路线：

$$\alpha_1 \xrightarrow{S} \beta_1 \xrightarrow{P} \alpha_2 \xrightarrow{S} \beta_2 \xrightarrow{P} \alpha_3 \xrightarrow{S} \beta_3 \xrightarrow{P} \alpha_4 \xrightarrow{S} \beta_4 \xrightarrow{P} \alpha_1$$

为形成可执行的搜索算法，需要明确每一步的输出选择。P 层的矩阵运算已经确定，而 S 层存在多个候选输出。以偏差尽可能大为目标，可以在候选中选择偏差最大的输出，这对应于一种贪心搜索思路。由于本任务只要求找到一条满足条件的路线，可以先采用这一方法。虽然贪心算法不保证全局最优，但可以从局部较优的选择出发，寻找满足要求的路线。

为进一步简化 P 置换带来的影响，可以先寻找该置换的不动点，即令 $P\beta = \beta$，等价于 $(P - I)\beta = 0$。若该矩阵不满秩，则存在非零解。观察前面的矩阵可知，P 的第一行第一列为 1，减去单位矩阵后，第一行全为 0，因此该矩阵不满秩。由此可以选择满足上述条件的 β，如下图所示。

```
[sage: I = identity_matrix(GF(2), 16)
[sage: M = Matrix(GF(2),M)
[sage: M = M - I
[sage: M.right_kernel()
Vector space of degree 16 and dimension 10 over Finite Field of size 2
Basis matrix:
[1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0]
[0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0]
[0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1]
```

例如，$\beta = 1000000000000000$，也就是 $\beta = (8, 0, 0, 0)$。进一步考虑迭代性质，可以先寻找一轮即可闭合的路线，再将其重复 4 次，从而避免直接搜索完整的四轮路线，即

$$\alpha_1 \xrightarrow{S} \beta_1 \xrightarrow{P} \alpha_1$$

还可以放宽 $P\beta = \beta$ 这一限制，直接要求

$$P(\beta_1) = \alpha_1$$

4.2.3 单轮迭代路线的搜索与结果

在上述约束下，β_1 可直接由 α_1 确定，从而简化搜索算法。此前限制活跃 S 盒的数量主要是为了方便计算；在仅搜索单轮迭代路线时，可以重新考虑这一限制。虽然可以从一个、两个活跃 S 盒的情形依次尝试，但全部非零输入掩码仅有 $2^{16} - 1$ 种，因此可直接

遍历全部 $2^{16}-1$ 个非零 α，覆盖不同活跃 S 盒数量的情况。随后通过 P^{-1} 计算 β，查询 LAT 并将各 S 盒的相关度相乘，选择绝对相关度最大的候选，再将该路线重复四轮。对应伪代码如算法 2所示。

算法 2 CipherFour 四轮迭代线性路线的搜索

输入： 4 比特 S 盒线性近似表 $\mathrm{LAT}[0..15, 0..15]$，16 比特置换 P
输出： 四轮迭代线性路线及其理论偏差

$\alpha^* \leftarrow 0,\ \beta^* \leftarrow 0,\ C^* \leftarrow 0$
for $\alpha = 1, \ldots, 2^{16}-1$ **do**
  $\beta \leftarrow P^{-1}(\alpha)$
  $C \leftarrow 1$
  for $j = 0, \ldots, 3$ **do**
    $a \leftarrow (\alpha \gg 4j)\&0xF$
    $b \leftarrow (\beta \gg 4j)\&0xF$
    $C \leftarrow C \cdot \mathrm{LAT}[a, b]/8$
  end for
  if $|C| > |C^*|$ **or** $(|C| = |C^*|$ **and** $\alpha > \alpha^*)$ **then**
    $\alpha^* \leftarrow \alpha$
    $\beta^* \leftarrow \beta$
    $C^* \leftarrow C$
  end if
end for
$C_4 \leftarrow (C^*)^4$
$\varepsilon_4 \leftarrow C_4/2$
$T \leftarrow [\alpha^*, \alpha^*, \alpha^*, \alpha^*, \alpha^*]$
return $T, \beta^*, \varepsilon_4$

代码运行结果如图 4所示。

```
wjy@wjydeMacBook-Air crypto-analyze % /usr/local/bin/python3 /Users/wjy/大三/实验
================================================================================
任务二：CipherFour 四轮迭代线性路线搜索
================================================================================
S盒（实验题目表1.1）：9 8 0 1 E B C D 3 2 5 7 F A 6 4
P置换（课程参考程序）：[0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15]
非零相关度的单轮固定点候选数量：3152

搜索得到的一条最优单轮固定点迭代路线：
  S层输入掩码 alpha = 0x8000
  S层输出掩码 beta  = 0x8000
  P(beta)           = 0x8000
  满足 P(beta) = alpha: True

逐轮信息（4轮，每轮重复同一组S层掩码）：
  第1轮：0x8000 --S--> 0x8000 --P--> 0x8000; C = -1/2 (-0.50000000)
  第2轮：0x8000 --S--> 0x8000 --P--> 0x8000; C = -1/2 (-0.50000000)
  第3轮：0x8000 --S--> 0x8000 --P--> 0x8000; C = -1/2 (-0.50000000)
  第4轮：0x8000 --S--> 0x8000 --P--> 0x8000; C = -1/2 (-0.50000000)

4轮输入掩码路线：
  0x8000 -> 0x8000 -> 0x8000 -> 0x8000 -> 0x8000

每个S盒的LAT贡献：
  S1: (a,b)=(8,8), LAT[8][8]=-4, C=-1/2
  S2: (a,b)=(0,0), LAT[0][0]=8, C=1
  S3: (a,b)=(0,0), LAT[0][0]=8, C=1
  S4: (a,b)=(0,0), LAT[0][0]=8, C=1

单轮相关度 C1      = -1/2 = -0.50000000
单轮偏差 epsilon1   = -1/4 = -0.25000000
4轮路线相关度 C4    = 1/16 = +0.06250000
4轮路线理论偏差     = 1/32 = +0.03125000
4轮路线绝对偏差     = 1/32 = 0.03125000
注：轮密钥可能改变偏差符号；此处给出不含密钥符号因子的路线偏差。
注：单条路线的偏差不等于线性包络的总偏差。
wjy@wjydeMacBook-Air crypto-analyze %
```

Figure 4: 任务二四轮迭代线性路线搜索结果

程序共找到 3152 个单轮相关度非零的候选，其中一条绝对相关度最大的路线满足

$$\alpha = 0x8000, \qquad \beta = 0x8000, \qquad P(0x8000) = 0x8000.$$

因此，该路线可迭代 4 轮，得到

$$0x8000 \longrightarrow 0x8000 \longrightarrow 0x8000 \longrightarrow 0x8000 \longrightarrow 0x8000.$$

对于每一轮，仅第一个 S 盒的输入、输出掩码非零，分别为 $(8, 8)$；其余 3 个 S 盒的掩码均为 $(0, 0)$。由任务一所得 LAT 可知

$$\mathrm{LAT}[8, 8] = -4, \qquad \mathrm{LAT}[0, 0] = 8.$$

故单轮相关度为

$$C_1 = \frac{-4}{8}\left(\frac{8}{8}\right)^3 = -\frac{1}{2},$$

相应的单轮偏差为 $\varepsilon_1 = C_1/2 = -1/4$。根据堆积引理（Piling-up Lemma），4 轮**单条线性路线**的相关度为

$$C_4 = \prod_{i=1}^{4} C_i = \left(-\frac{1}{2}\right)^4 = \frac{1}{16}.$$

从而得到不计密钥常数项时的理论偏差

$$\varepsilon_4 = \frac{C_4}{2} = \frac{1}{32} = 0.03125.$$

上述结果来自单轮路线重复四次的搜索，仍需进一步判断其是否为全部四轮迭代路线中的最优解。例如，还可以考虑将两条两轮路线拼接，或直接构造一条四轮闭合路线。搜索这些路线时，活跃 S 盒的数量再次影响分析规模，因为中间状态无法像单轮情形一样由 P 置换直接确定。针对这一问题，我们借助 AI 进行进一步分析，结果如下。



用时 4m 55s >

全局最优四轮闭环是

$$8000 \to 8000 \to 8000 \to 8000 \to 8000.$$

按最高位为第 0 位编号，$P[i]$ 表示把源位 i 移到目标位 $P[i]$；例如 $P(1234) = 016A$，且 $P(8000) = 8000$。下表采用完整 Walsh LAT，相关度为 LAT 除以 16。

轮次	输入掩码	S 盒输出掩码	P 后掩码	四个 S 盒的 LAT（高位到低位）	本轮相关度
1	8000	8000	8000	$(-8, 16, 16, 16)$	$-1/2$
2	8000	8000	8000	$(-8, 16, 16, 16)$	$-1/2$
3	8000	8000	8000	$(-8, 16, 16, 16)$	$-1/2$
4	8000	8000	8000	$(-8, 16, 16, 16)$	$-1/2$

因此路线相关度为 $C = (-1/2)^4 = 1/16$，偏差绝对值为 $|\varepsilon| = |C|/2 = 1/32$。若沿用前文的半 Walsh LAT 约定，表中每项除以 2，即活跃 S 盒的 `LAT[8][8] = -4`。

完整的标准库 Python 验证脚本 对全部 65,535 个非零起始掩码计算四轮开放路线的精确上界。只有 `00BB`、`0660`、`0BB0`、`B00B`、`B0B0`、`B0BB`、`BB0B` 七个起点的上界可能超过 $1/16$；脚本继续检查它们所有可能超越该值的闭环，结果均未超越。搜索允许任意数量的活跃 S 盒，也没有禁止中途重复状态，因此 $1/32$ 是所求的全局最大偏差绝对值。

星期日16:16

该辅助分析给出的全局最优路线与本实验搜索所得路线一致。

4.3 任务三

题目给出的 PHILOX-2W 算法是一种基于 Feistel 结构的分组密码，其左右分支长度均为 $W = 8$bit，乘法常数为 $M = 0x9B$。实验要求分析输入掩码分别为 $(0, 1)$、$(1, 0)$、$(0, 2)$、$(2, 1)$ 时的一轮线性掩码传播情况，并找出每组输入掩码对应的偏差绝对值最大的前 5 个输出掩码及其偏差。

根据题目给出的轮函数，设输入状态为 (L, R)，轮密钥为 K，一轮输出状态为 (L', R')，则有

$$L' = \text{mullo}_W(R) R' = \text{mulhi}_W(R) \oplus L \oplus K$$

其中

$$\text{mulhi}_W(R) = \left\lfloor \frac{R \cdot M}{2^W} \right\rfloor, \qquad \text{mullo}_W(R) = (R \cdot M) \bmod 2^W$$

设输入掩码为 $\alpha = (\alpha_L, \alpha_R)$，输出掩码为 $\beta = (\beta_L, \beta_R)$，则对应的线性近似式为

$$\alpha_L \cdot L \oplus \alpha_R \cdot R \oplus \beta_L \cdot L' \oplus \beta_R \cdot R'$$

将轮函数代入上述表达式，整理可得

$$(\alpha_L \oplus \beta_R) \cdot L \oplus \alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \beta_R \cdot \text{mulhi}_W(R) \oplus \beta_R \cdot K$$

将含 L 的项与其余项分离，并将后者记为 $f(R, K)$，则线性近似式可写为

$$(\alpha_L \oplus \beta_R) \cdot L \oplus f(R, K)$$

其中

$$f(R, K) = \alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \beta_R \cdot \text{mulhi}_W(R) \oplus \beta_R \cdot K$$

由于 L, R 相互独立，可以分别讨论不同掩码关系下的情形。输入随机时，分支 L 在 $\mathbb{F}_2^8$ 上均匀分布。

4.3.1 $\alpha_L \neq \beta_R$

此时有

$$\Pr((\alpha_L \oplus \beta_R) \cdot L = 0) = \Pr((\alpha_L \oplus \beta_R) \cdot L = 1) = \frac{1}{2}$$

于是

$$\Pr(f(R, K) = 0) = \Pr(f(R, K) = 1) = \frac{1}{2}$$

根据堆积引理，可得 $\Pr((\alpha_L \oplus \beta_R) \cdot L \oplus f(R, K) = 0) = \frac{1}{2}$。因此，该情况下的线性偏差为 0，属于平凡情况。

4.3.2 $\alpha_L = \beta_R$

此时，线性近似式与 L 无关，可化为

$$\alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \alpha_L \cdot \text{mulhi}_W(R) \oplus \alpha_L \cdot K$$

该式仍包含密钥 K。在计算固定密钥下的线性近似表时，仅输入发生变化，因此 $\alpha_L \cdot K$ 在线性近似表的每一项中均为常量，只可能取 0 或 1，不影响线性偏差的绝对值。若 $\alpha_L \cdot K = 0$，则线性近似式取 0 的输入个数，等于去掉常数项后表达式取 0 的输入个数，即

$$\text{LAT}(\alpha, \beta) = \#\{R | \alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \alpha_L \cdot \text{mulhi}_W(R) = 0\} - 2^7$$

若 $\alpha_L \cdot K = 1$，则线性近似式取 0 的输入个数，等于去掉常数项后表达式取 1 的输入个数。因此，可用输入总数 2^8 减去该表达式取 0 的输入个数，得到

$$\text{LAT}(\alpha, \beta) = 2^8 - \#\{R | \alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \alpha_L \cdot \text{mulhi}_W(R) = 0\} - 2^7$$

整理得

$$\text{LAT}(\alpha, \beta) = 2^7 - \#\{R | \alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \alpha_L \cdot \text{mulhi}_W(R) = 0\}$$

与前一情形相比，表项仅相差一个符号，其绝对值相同。因此，为便于计算，以下取 $\alpha_L \cdot K = 0$。

程序首先遍历题目给出的 4 组输入掩码 $\alpha = (\alpha_L, \alpha_R)$，根据上述条件确定输出掩码右分支 $\beta_R = \alpha_L$，再遍历全部 2^8 种输出左分支掩码 β_L。对于每组输入、输出掩码，遍历全部 2^8 种右分支输入 R，统计对应线性近似式取 0 的输入个数 $N(\alpha, \beta)$，并计算线性偏差

$$\varepsilon(\alpha, \beta) = \frac{N(\alpha, \beta)}{2^8} - \frac{1}{2}.$$

最后将所有输出掩码按照线性偏差的绝对值降序排列，选取前 5 个输出掩码及其偏差。记 parity(v) 为 v 的二进制表示中 1 的个数模 2，& 表示按位与运算，对应伪代码如算法 3所示。

算法 3 PHILOX-2W 一轮线性掩码传播分析

输入： $W = 8$，$M = 0x9B$，输入掩码集合 $\mathcal{A} = \{(0,1), (1,0), (0,2), (2,1)\}$
输出： 各输入掩码对应的前 5 个输出掩码及其线性偏差

for $(\alpha_L, \alpha_R) \in \mathcal{A}$ **do**
  $T \leftarrow \emptyset$
  $\beta_R \leftarrow \alpha_L$
  for $\beta_L = 0, \ldots, 255$ **do**
    if $(\beta_L, \beta_R) = (0, 0)$ **then**
      continue
    end if
    $N \leftarrow 0$
    for $R = 0, \ldots, 255$ **do**
      $H \leftarrow \lfloor R \cdot M / 2^8 \rfloor$
      $D \leftarrow (R \cdot M) \bmod 2^8$
      $p \leftarrow \text{parity}(\alpha_R \,\&\, R)$
      $q \leftarrow \text{parity}(\beta_L \& D) \oplus \text{parity}(\beta_R \wedge H)$
      if $p = q$ **then**
        $N \leftarrow N + 1$
      end if
    end for
    $\varepsilon \leftarrow N/256 - 1/2$
    $T \leftarrow T \cup \{(\beta_L, \beta_R, \varepsilon)\}$
  end for
  按 $|\varepsilon|$ 降序排列 T
  Result$[\alpha] \leftarrow$ Top5(T)
end for
return Result

代码运行结果如图 5所示。

```
wjy@wjydeMacBook-Air crypto-analyze % /usr/local/bin/python3 /Users/wjy/大三/实验
================================================================================
PHILOX-2W 一轮线性掩码传播分析
================================================================================
W = 8
M = 0x9B
K = 0x00

--------------------------------------------------------------------------------
输入掩码 alpha = (0x00, 0x01)
--------------------------------------------------------------------------------
Rank    Output Mask          Bias             Absolute Bias
1       (0x01, 0x00)      +0.50000000       0.50000000
2       (0x02, 0x00)      +0.00000000       0.00000000
3       (0x03, 0x00)      +0.00000000       0.00000000
4       (0x04, 0x00)      +0.00000000       0.00000000
5       (0x05, 0x00)      +0.00000000       0.00000000

--------------------------------------------------------------------------------
输入掩码 alpha = (0x01, 0x00)
--------------------------------------------------------------------------------
Rank    Output Mask          Bias             Absolute Bias
1       (0x5A, 0x01)      -0.14453125       0.14453125
2       (0xDA, 0x01)      +0.11328125       0.11328125
3       (0xDB, 0x01)      +0.11328125       0.11328125
4       (0xD2, 0x01)      +0.10546875       0.10546875
5       (0x5B, 0x01)      +0.08984375       0.08984375

--------------------------------------------------------------------------------
输入掩码 alpha = (0x00, 0x02)
--------------------------------------------------------------------------------
Rank    Output Mask          Bias             Absolute Bias
1       (0x03, 0x00)      +0.50000000       0.50000000
2       (0x01, 0x00)      +0.00000000       0.00000000
3       (0x02, 0x00)      +0.00000000       0.00000000
4       (0x04, 0x00)      +0.00000000       0.00000000
5       (0x05, 0x00)      +0.00000000       0.00000000

--------------------------------------------------------------------------------
输入掩码 alpha = (0x02, 0x01)
--------------------------------------------------------------------------------
Rank    Output Mask          Bias             Absolute Bias
1       (0xB6, 0x02)      +0.17578125       0.17578125
2       (0xA4, 0x02)      +0.16015625       0.16015625
3       (0xF4, 0x02)      -0.13671875       0.13671875
4       (0xB4, 0x02)      -0.11328125       0.11328125
5       (0xBC, 0x02)      -0.10546875       0.10546875

================================================================================
全部输入掩码分析完成!
```

Figure 5: PHILOX-2W 一轮线性掩码传播分析结果

具体结果如表 1所示。

Table 1: PHILOX-2W 一轮线性掩码传播分析结果

输入掩码 α	Rank	输出掩码 β	Bias	Absolute Bias
$(0x00, 0x01)$	1	$(0x01, 0x00)$	$+0.50000000$	0.50000000
	2	$(0x02, 0x00)$	$+0.00000000$	0.00000000
	3	$(0x03, 0x00)$	$+0.00000000$	0.00000000
	4	$(0x04, 0x00)$	$+0.00000000$	0.00000000
	5	$(0x05, 0x00)$	$+0.00000000$	0.00000000
$(0x01, 0x00)$	1	$(0x5A, 0x01)$	-0.14453125	0.14453125
	2	$(0xDA, 0x01)$	$+0.11328125$	0.11328125
	3	$(0xDB, 0x01)$	$+0.11328125$	0.11328125
	4	$(0xD2, 0x01)$	$+0.10546875$	0.10546875
	5	$(0x5B, 0x01)$	$+0.08984375$	0.08984375
$(0x00, 0x02)$	1	$(0x03, 0x00)$	$+0.50000000$	0.50000000
	2	$(0x01, 0x00)$	$+0.00000000$	0.00000000
	3	$(0x02, 0x00)$	$+0.00000000$	0.00000000
	4	$(0x04, 0x00)$	$+0.00000000$	0.00000000
	5	$(0x05, 0x00)$	$+0.00000000$	0.00000000
$(0x02, 0x01)$	1	$(0xB6, 0x02)$	$+0.17578125$	0.17578125
	2	$(0xA4, 0x02)$	$+0.16015625$	0.16015625
	3	$(0xF4, 0x02)$	-0.13671875	0.13671875
	4	$(0xB4, 0x02)$	-0.11328125	0.11328125
	5	$(0xBC, 0x02)$	-0.10546875	0.10546875

偏差为 0.5 意味着相应线性近似式取 0 的概率为 1，即该关系必然成立，因此可用作有效的区分器。

A 实验代码

完整代码详见附件。

B 实验报告完整地址

实验完整代码、实验报告 LaTeX 源代码见仓库：https://github.com/infinitepwn/crypto-analyze.git

C AI 使用情况

本实验使用 AI 辅助编程和实验报告的排版、错别字检查，实验主体流程设计、结果分析和实验报告的撰写未使用 AI。